ISTANBUL KÜLTÜR UNIVERSITY

**Department of Computer Engineering**

# SEN0401 Special Topics: Blockchain
# Midterm Exam

| | |
|---|---|
| **Student Name** | **:** |
| **Student Identity Number** | **:** |

**Academic Year** : **2026-2027**

**Term** : **Fall**

**Date & Time** : **____ / ____ / 2026 ____ : ____ (fill in)**

**Duration** : **60 Minutes**

**Instructor** : **Dr. Yusuf ALTUNEL**

| Question | Grade |
|---|---|
| 1 | /25 |
| 2 | /25 |
| 3 | /25 |
| 4 | /25 |
| 5 (bonus) | /10 |
| Total | /100 + bonus |

**INSTRUCTIONS**

- Check that the form letter and the form code printed on the question pages are the same as those printed on your answer sheet (Form A, form code 30015).
- Write your name and your student identity number in the boxes above and on the answer sheet before starting the test.
- Questions 1 to 4 are answered on the **separate answer sheet, which is read by a machine**. Use a dark (black or blue) pen and **fill each chosen mark completely**. To cancel an answer, cross its mark out with an X, then fill the mark you now choose. Do not use correction fluid. **Question 5 (the bonus) is answered with a pen on the lines of the question pages and is marked by hand.**
- Write only inside the boxes and marks of the answer sheet, and on the lines of Question 5. You may use the blank areas of the question pages for scratch work; do not use any additional scratch paper.
- Writing that is unclear or outside the boxes **may not be read and will not be graded**.
- Allowed on the table: **the question paper, the answer sheet, a pen and an eraser**, and: _____ (instructor to fill in, for example a calculator, a note sheet, or nothing). Everything else should be cleared. **Do not share your belongings** with other students during the exam.
- Be particularly aware that the **use or the presence** of cell phones, laptops, or other networked devices is **FORBIDDEN** and will be considered a **cheating attempt**.
- You are **not allowed to communicate with other students** even for requesting stationery material such as pen, pencil, eraser, etc., unless authorized explicitly by the instructor.

| Question (Q) | Q1 | Q2 | Q3 | Q4 | Q5 |
|---|---|---|---|---|---|
| Learning Outcomes | 1, 2, 3, 4, 5 | 1, 3, 4, 5, 6 | 2, 3, 5, 6 | 2, 3, 4, 6 | 1, 2 |
| Program Outcomes | ___ | ___ | ___ | ___ | ___ |

**WARNING:** During the examination, please do not attempt to **cheat**. Suspected cheating will result in a **ZERO** on your exam. As cheating in examination is a disciplinary action, you will be subject to disciplinary hearing.

# SEN0401 Special Topics: Blockchain - Midterm exam, Form A

**Form code 30015** (it must equal the form code printed on your answer sheet). Chapters 1, 2, 3, 4, 5, 6. 60 minutes. Questions 1 to 4 (25 points each) are answered on the separate answer sheet; Question 5 is a 10-point bonus, written on the lines below it. Give ONE answer per row unless the question says otherwise.

## Question 1 - Reading and choice (25 points)

Read the situation and its tips. Mark the one statement that is correct (letters A to D) on the answer sheet.

**1a.** Dana's coins were sent to the address of the compressed public key of her private key. Her paper backup holds that key in Wallet Import Format (WIF), the text form for importing a private key; a WIF for a compressed key ends with an extra byte 01, but hers does not. A new wallet imports it and shows zero balance. Tips: compressed public key: the shorter form of a public key; a private key has two public key forms and so two addresses, and coins sent to one are not seen at the other. Which diagnosis fits, and which check before funding would have caught it? (4 points)

**A.** The wallet took the key as uncompressed and watches its other address; comparing the first address after a test restore would have caught it.

**B.** The coins went to an address with no key behind it; checking the address again would have caught it.

**C.** The wallet looks on another network; a tiny first payment and a day's wait would have caught it.

**D.** The missing byte changed the private key; only a second copy of the paper would have caught it.

**1b.** Chance, in percent, that an attacker with a share of the hash power ever catches up when a payment has z confirmations: for 10 percent, z = 1 gives 20.5, z = 2 gives 5.1, z = 3 gives 1.3 and z = 6 gives 0.02; for 30 percent, z = 1 gives 62.8, z = 3 gives 32.5, z = 6 gives 13.2, z = 9 gives 5.5 and z = 10 gives 4.2. A shop accepts a payment only when the chance of a reversal is below 5 percent. Tips: confirmation: each further block built on the block that holds a payment; hash power: computing power used to add blocks; attacker: someone who rebuilds the latest blocks to take a payment back. Which statement follows from these figures? (4 points)

**A.** Three confirmations are enough against the 30 percent attacker, because the chance is below 5 percent at z = 3.

**B.** A shop needs three confirmations against the 10 percent attacker but ten against the 30 percent one.

**C.** Six confirmations are equally safe against either attacker, because each divides the chance equally.

**D.** Two confirmations are enough against the 10 percent attacker, because z = 2 is below 5 percent.

**1c.** A node reads the configuration file and is started with the command line shown. Tips: prune: deletes old block files to save disk; txindex: indexes all transactions; it needs all blocks, which pruning deletes; dbcache and bitcoind: the memory cache size and the node program; command line: options typed at the start; they replace the same options of the file for that run. What happens, and why? (3 points)

```
# bitcoin.conf
prune=550
txindex=1
dbcache=450
# the command line
bitcoind -txindex=0 -dbcache=2048
```

**A.** It starts with pruning and the index on, because the refusal covers only the command line.

**B.** It refuses to start: the file asks for pruning and the index together, which clash.

**C.** It starts with the index on and a cache of 450, because the file replaces the command line.

**D.** It starts with pruning 550, no index and cache 2048: the command line replaces the file, and no index removes the clash.

**1d.** Two stories about the exchange Mt. Gox in 2014. On 28 February it filed for bankruptcy after admitting that about 850,000 bitcoin were gone. Three weeks later it said it had found about 200,000 bitcoin in an old-format wallet file it believed was empty. Tips: Mt. Gox: a Tokyo bitcoin exchange; keys: secret numbers that let their holder spend coins; wallet file: stores a wallet's keys; bankruptcy: a court closes a company that cannot pay. Which statement says correctly what each story teaches? (4 points)

**A.** Filing: the network let the exchange spend the same coins twice. Old file: miners held the coins back for a fee.

**B.** Filing: customers lost their own keys. Old file: backups expire, so an old file can be thrown away.

**C.** Filing: whoever holds the keys holds the coins, and customers held none. Old file: an untested file is not an empty one.

**D.** Filing: an untested file is not an empty one. Old file: whoever holds the keys holds the coins.

**1e.** A node's configuration file holds an rpcauth line, and a pay to public key hash (P2PKH) output script (the lock of the older address format) holds a 20-byte hash of a public key. Both keep a hash, not the secret. Tips: rpcauth: the node setting (remote procedure call authentication) that checks passwords of programs calling the node; hash: a one-way fingerprint; salted: mixed with random data, so equal passwords give different hashes; commitment: fixes a secret now; its owner reveals it later. Which statement compares them correctly, and why do both keep a hash? (4 points)

**A.** A leak reveals the password, because rpcauth is only an encoding; the P2PKH hash comes with the key.
**B.** The rpcauth hash is a commitment revealed when coins are spent; the P2PKH hash is salted, so a leak reveals no key.
**C.** A leak reveals no password, because rpcauth is salted; the P2PKH hash is a commitment, and the key appears only when coins are spent.
**D.** Both hashes are salted checksums that catch typos, and both secrets appear when a node reads them.

**1f.** A wallet shows a draft batching payment: inputs of 400,000 and 250,000 satoshis, outputs of 300,000, 200,000, 100,000 and 29,000 satoshis. A colleague says that every node will refuse it because inputs and outputs are not equal. Tips: satoshi: one hundred-millionth of a bitcoin; batching: one transaction paying several people; node: a computer that checks the rules; consolidation: many small coins gathered into one; change: the part of the inputs that returns to the sender. Which reply is correct, and why? (3 points)

**A.** Yes: four outputs make it a consolidation, which must spend exactly its inputs.
**B.** No: the 21,000 satoshis left over are the miner's fee, the difference between inputs and outputs; no field records it.
**C.** No: the 21,000 satoshis return to the wallet as change, so no fee is paid.
**D.** Yes: the ledger must balance exactly, so one output must be lowered by 21,000.

**1g.** Lena's phone is stolen. From her twelve-word recovery code a new phone shows different addresses, an empty balance and no labels. She had set a passphrase and had exported her labels in the Bitcoin Improvement Proposal 329 (BIP329) format, a standard label file. Tips: recovery code: words that write down the secret a wallet is built from; passphrase: an extra secret; the same words with another passphrase open another wallet; labels: notes on addresses and payments; gap limit: unused keys in a row after which a recovery scan stops. What is the most complete explanation and the fix? (3 points)

**A.** A passphrase forks the wallet and labels cannot be derived from the seed; she must enter the passphrase and import the labels file.
**B.** The recovery code was on the stolen phone, so the thief has spent the coins.
**C.** Labels come from the seed and were lost with the phone; the passphrase only changes the spelling.
**D.** The gap limit stopped the scan too early; a larger one would bring back the balance and the labels.

## Question 2 - Calculation and tracing (25 points)

Work out each number (by calculation, or by following the program) and write it, at most five characters, in the boxes of the answer sheet.

**2a.** A wallet that is recovered from its code scans its keys in order and stops after gap unused keys in a row. The wallet received payments on keys 0, 3, 12 and 30. The program adds up the numbers of the keys that the scan finds. What does it print? (4 points)

```
used = {0, 3, 12, 30}
gap, run, found = 10, 0, []
for i in range(40):
    if i in used:
        found.append(i)
        run = 0
    else:
        run += 1
        if run == gap:
            break
print(sum(found))
```

Printed result: sheet row 2a, one digit per box.

**2b.** A toy chain pays a block subsidy of 40 coins. Every 50 blocks the subsidy is halved by a right shift, so any fraction is dropped, and once it reaches zero no more coins are created. The same arithmetic limits the real supply to about 21 million. Tips: block subsidy: the new coins that each block creates for its miner; right shift: moving every binary digit one place to the right, which halves a whole number and drops the fraction (for example 5 becomes 2). How many coins does the subsidy create in total over the whole life of the chain? (3 points)
Sheet row 2b, one digit per box. Unit: coins.

**2c.** A script drops the release candidates from a list of release tags and compares the final releases sorted as text with the same releases sorted by version number. It adds up how far each tag moves between the two orders. Tips: release tag: the name of one version of a program, such as v3.1; release candidate: a test version before the final one; its tag carries rc; sorted as text: letter by letter, so "v28.0" comes before "v3.1". What does it print? (3 points)

```
tags = ['v28.0', 'v3.1', 'v28.0rc1',
        'v4.0', 'v24.2', 'v9.1']
final = [t for t in tags if 'rc' not in t]
def key(t):
    return [int(p) for p in t[1:].split('.')]
a = sorted(final)
b = sorted(final, key=key)
move = 0
for t in final:
    move += abs(a.index(t) - b.index(t))
print(move)
```

Printed result: sheet row 2c, one digit per box.

**2d.** A search for a vanity address needs, on average, a number of tries for one chosen letter after the leading 1. Nobody can shorten the search. Tips: vanity address: an address chosen to start with letters its owner likes; 58 symbols: an address is written with 58 symbols, so each further chosen letter makes a match 58 times less likely. By what factor is the average number of tries larger when the search asks for three chosen letters instead of one? (3 points)
Sheet row 2d, one digit per box. Unit: times.

**2e.** The program reads five sequence values (a 32-bit number in each input). A value signals opt-in replacement when it is below 0xfffffffe and carries a relative timelock when bit 31 is clear. It adds up the low 16 bits of the values that do both. Tips: opt-in replacement: the unconfirmed payment may be replaced by one with a higher fee; relative timelock: the spent output must first wait a set time; bit 31: the highest of the 32 bits; clear means 0; 0x: starts a hexadecimal number. What does it print? (3 points)

```
seqs = [0xffffffff, 0xfffffffd, 0x0000001e,
        0x80000010, 0x00400005]
total = 0
for s in seqs:
    replace = s < 0xfffffffe
    timed = (s >> 31) == 0
    if replace and timed:
        total += s & 0xffff
print(total)
```

Printed result: sheet row 2e, one digit per box.

**2f.** Trades discover the price of bitcoin. On one day a price service sees three trades of bitcoin: 1 bitcoin at 62,000 dollars, 2 bitcoin at 63,000 dollars, and 1 bitcoin at a price that is not shown. The service publishes a volume-weighted figure for the day of 63,250 dollars per bitcoin. Tips: volume-weighted figure: an average in which each trade counts in proportion to the amount traded, so a large trade moves it more than a small one. At what price, in dollars per bitcoin, did the third trade happen? (3 points)
Sheet row 2f, one digit per box. Unit: dollars per bitcoin.

**2g.** The function cs_encode writes a count in the CompactSize encoding: a count below 253 takes 1 byte, up to 65535 takes 3 bytes, up to 4294967295 takes 5 bytes, anything larger takes 9 bytes. The program measures four counts that sit at the edges where the encoding changes length. What does it print? (3 points)

```
values = [252, 253, 65536, 4294967295]
sizes = [len(cs_encode(n)) for n in values]
print(sum(sizes))
```

Printed result: sheet row 2g, one digit per box.

**2h.** A recovery code has 18 words. Each word carries 11 bits, and the checksum is one bit for every 32 bits of entropy. Tips: recovery code: words that write down the secret a wallet is built from; entropy: the amount of randomness in the secret, in bits; checksum: extra bits that catch a mistyped word. How many bits of entropy does the code hold? (3 points)
Sheet row 2h, one digit per box. Unit: bits.

## Question 3 - Ordering and matching (25 points)

Follow the instruction under each part. The points of a part divide equally over its steps or pairs: every correct step or pair earns its share.

**3a.** Match each story (left) with the lesson (right) that it teaches. Two lessons are not used. (9 points; 2.25 for each correct pair.) On the answer sheet, rows 3a1 to 3a4: fill the letter of the answer.

**1)** A programmer, paid 7,002 bitcoin in 2011, lost the password of the drive with his keys; two guesses are left.
**2)** In block 74,638 (15 August 2010) two huge outputs summed to a negative number; 184 billion bitcoin appeared.
**3)** On 7 April 2014 the reference software got a new lead maintainer.
**4)** On 10 February 2014 an exchange blamed transaction malleability (changing a transaction's identifier, not the payment) for missing bitcoin; researchers checked a year of traffic.

**A.** A transaction has no fee field: the fee is inputs minus outputs.
**B.** A signed amount needs a range check (zero up to the maximum) on each output, not only on the total.
**C.** A double spend is refused by a shared rule, and no bank decides.
**D.** Memory is not a backup: test a recovery code by restoring it.
**E.** A reference implementation (the program that defines the rules) needs a team and public review.
**F.** A stated cause is tested against data: malleability changes the transaction identifier (txid).

**3b.** Put the five steps of one payment in the order in which they happen: write 1 for the first and 5 for the last. Each step needs the result of the step before it. (8 points; 1.6 for each step in the right place.) The steps are numbered 1) to 5) as printed. On the answer sheet, rows 3b1 to 3b5: for each step fill the place (1 = first, 5 = last) at which it happens.

**1)** The wallet takes whole coins from its unspent outputs (coins no later payment has used) until they cover the payment and the fee.
**2)** The wallet writes the payee output and the change output (the part that returns to the sender) and leaves the fee as the difference.
**3)** A node (a computer that checks the rules) checks the transaction and relays it, that is, passes it on to its peers.
**4)** A miner finds a valid nonce (the number varied in the search for a block) and adds a block that holds the transaction.
**5)** The wallet signs every input (proves that it may spend the coin the input names).

**3c.** Match each need (left) with the setting (right) that serves it. Each response says what the setting does. Two responses are not used. (8 points; 2 for each correct pair.) On the answer sheet, rows 3c1 to 3c4: fill the letter of the answer.

**1)** A volunteer runs a validating node (a node that checks all blocks and transactions) on a laptop with little disk space and does not need old blocks.
**2)** A block-explorer service (a website for looking up blocks and transactions) must find any transaction by its identifier.
**3)** The operator wants a message, usually by email, when the node meets an emergency.
**4)** A node on a thin connection must limit how many other nodes it accepts.

**A.** txindex: indexes all transactions (needs all blocks)
**B.** alertnotify: sends emergency alerts, usually by email
**C.** datadir: chooses where all blockchain data is stored
**D.** maxconnections: limits connecting nodes, to save bandwidth
**E.** maxmempool: limits memory for waiting transactions
**F.** prune: deletes old blocks to save disk space

## Question 4 - Fill in the sentence (25 points)

Each group has its own bank. Choose for each blank the entry that makes the sentence true; each entry says what it means. Write the bank code.

**4a-4b.** Fill each blank with the two-digit CODE of the matching bank entry (sheet rows 4a to 4b). Each entry is used once; some stay unused.

| | |
|---|---|
| **13** more (a larger amount) | **21** wtxid (the witness transaction identifier, a hash that also covers the witness) |
| **37** less (a smaller amount) | **45** txid (the transaction identifier, a hash that ignores the witness) |

**4a.** Segregated witness (SegWit) keeps the signatures outside the data that gives the transaction identifier, so a changed signature changes the witness transaction identifier (wtxid) but not the ____. (5 points)
**4b.** A block is filled by weight: each byte counts four units, but the witness (the signatures) and the marker and flag (two bytes that announce a witness) count one. A payment with many witness bytes therefore uses ____ block space than one with the same number of other bytes. (4 points)

**4c-4d.** Fill each blank with the two-digit CODE of the matching bank entry (sheet rows 4c to 4d). Each entry is used once; some stay unused.

| | |
|---|---|
| **13** more (a larger amount) | **21** unspent (not yet used as an input by a later transaction) |
| **37** less (a smaller amount) | **45** spent (already used as an input by a later transaction) |

**4c.** The fee is taken from what would otherwise return to the wallet as change. With the same coin and the same payment, a higher fee rate therefore leaves ____ change. (4 points)
**4d.** A spend removes the outputs it uses from the set of ____ outputs and adds the outputs it creates. Wallets and nodes keep exactly that set, because only its members can be spent. (4 points)

**4e-4f.** Fill each blank with the two-digit CODE of the matching bank entry (sheet rows 4e to 4f). Each entry is used once; some stay unused.

| | |
|---|---|
| **13** salt (random data mixed in before hashing) | **21** checksum (a few extra bytes that catch typing mistakes) |
| **37** 256-bit hash (the hash function Secure Hash Algorithm 256-bit (SHA256), whose result is 256 bits long) | **45** commitment (a value that fixes a secret now and lets its owner reveal it later) |

**4e.** A pay to public key hash (P2PKH) output holds only a 20-byte hash of the public key; the key stays hidden until the coins are spent. The hash is a ____ to the key. (4 points)
**4f.** A 160-bit hash gives only about 80 bits of protection against collisions (two different inputs with the same result). That is why the newer segregated witness (SegWit) scripts use the ____ instead. (4 points)

## Question 5 - Short explanation (bonus) (10 points, bonus)

Bonus question, marked by hand. Answer in two or three sentences in the lines under the question. Each tip says what a term means.

**5a.** In two or three sentences, explain why mining is called a search that nobody can shortcut, and why it is still easy for every other node to check a block. (5 points)
**Tips.** mining: the search for a number that makes the hash of a block header small enough; hash: a one-way fingerprint: easy to compute, practically impossible to reverse; node: a computer that checks every rule.

_____
_____
_____
_____
_____
_____

**5b.** In two or three sentences, explain who refuses a double spend (the same coin offered to two people) in Bitcoin when no bank stands in the middle, and why that makes the history costly to change. (5 points)
**Tips.** double spend: the same coin offered to two different people; node: a computer that checks every rule; proof of work: the costly search that every new block must contain.

_____
_____
_____
_____
_____
_____